TM
TESTMAKER

Avaliação: Arquitetura Orientada a Servicos

Prova · 2026

NOTA

Valor: 20,0 pts

ALUNO(A) ..... Nº .....

TURMA ..... PROFESSOR(A) Mateus Redivo DATA ..... / ..... / .....

INSTRUÇÕES

Leia atentamente cada questão antes de responder. Utilize **caneta azul ou preta**. **Não rasure** o cartão-resposta: preencha completamente a bolha correspondente.

01 Escreva o Javadoc do método abaixo, documentando o que ele faz, o parâmetro, o retorno e a exceção: **1,0 pt**

```
public ProdutoDTO buscar(Long id) {
    return produtoRepository.findById(id)
            .map(ProdutoMapper::toDTO)
            .orElseThrow(() -> new
            ProdutoNaoEncontradoException(id));
}
```

02 Considere uma versão modificada do método de atualização de um Service sem @Transactional, em que a linha de salvar foi removida: **1,0 pt**

```
public Optional<ProductDTO> updateProduct(Long
id, ProductDTO dto) {
    return productRepository.findById(id).map(product
    -> {
        product.setName(dto.getName());
        product.setDescription(dto.getDescription());
        product.setValue(dto.getValue());
        product.setQuantity(dto.getQuantity());
        return ProductMapper.toDTO(product);
        // productRepository.save(product); foi
        removida
    });
}
```

Sem a chamada a productRepository.save(product), o que acontece com as alterações feitas no produto?

A) As alterações são salvas sozinhas, pois o JPA detecta qualquer mudança em entidade buscada do banco

B) O produto é removido do banco, pois a falta do save() é interpretada como exclusão

C) O método lança uma exceção, pois a entidade não está mais ligada a uma transação ativa

D) As alterações ficam só no objeto em memória: a resposta mostra os valores novos, mas o banco não muda

E) O Hibernate impede a alteração dos campos até que a entidade seja salva de novo

03 Veja o código a seguir: **1,0 pt**

```
public class PedidoView {
    public void salvarPedido(Pedido pedido) {
        PedidoRepository repository = new
        PedidoRepository();
        repository.salvar(pedido);
    }
}
```

Qual o principal problema arquitetural?

A) O código está correto, pois a View só repassa a ação do usuário para quem sabe salvar

B) A View não deveria receber um Pedido, pois não pode conhecer o Model

C) O método deveria se chamar salvarNoBanco, para deixar a persistência explícita

D) O Repository deveria ser estático, para a View não precisar instanciá-lo

E) A View acessa a persistência diretamente, sem passar pelo Controller

04 Considere as afirmações sobre uma API em camadas com Spring Boot: **1,0 pt**

I. A anotação @Valid no parâmetro do Controller é o que faz o Spring aplicar as validações declaradas no DTO.

II. O Service pode devolver a entidade ao Controller, desde que o Controller não altere nenhum campo dela.

III. Uma exceção lançada pelo Service pode virar resposta HTTP em um @RestControllerAdvice, sem try/catch no Controller.

IV. O Controller só precisa conhecer o Service, e é o Service quem conhece o Repository.

Estão corretas:

A) I, II, III e IV

B) Apenas I, III e IV

C) Apenas II, III e IV

D) Apenas I, II e IV

E) Apenas I e III

05 O serviço de clientes precisa trocar o campo nome por nomeCompleto na resposta, mas dez sistemas já usam o campo nome. Qual é a abordagem adequada? **1,0 pt**

A) Publicar uma versão nova (/v2) e manter a /v1 até os consumidores migrarem

B) Trocar o campo direto e avisar as equipes por e-mail no mesmo dia

C) Criar um serviço novo com outro nome e desligar o antigo na mesma hora

D) Pedir que os dez sistemas passem a ler o nome direto no banco de dados

E) Fazer a troca de madrugada, quando os outros sistemas têm menos acessos

06 Considere o campo abaixo em um DTO: **1,0 pt**

```
@NotBlank(message = "Nome e obrigatorio")
@Size(min = 3, max = 50, message = "Nome deve
ter entre 3 e 50 caracteres")
private String nome;
```

Se uma requisição enviar "nome": "Al" (2 caracteres), o que acontece na validação?

A) Passa normalmente, pois @NotBlank já garante que o campo não está vazio

B) Falha nas duas validações ao mesmo tempo, com dois erros idênticos

C) A requisição é aceita, mas o valor é cortado para caber no limite

D) Falha só no @Size, pois "Al" não é vazio, mas tem menos de 3 caracteres

E) Falha só no @NotBlank, pois "Al" é considerado em branco

07 Analise o código abaixo: **1,0 pt**

```
public class PedidoController {
    public void aplicarDesconto(Pedido pedido) {
        if (pedido.getValor() > 500) {
            pedido.setValor(pedido.getValor() * 0.9);
        }
    }
}
```

Qual alternativa está correta?

A) O desconto deveria ficar na View, que mostra o valor final ao usuário

B) A regra de desconto é de negócio e deveria estar no Service, não no Controller

C) O problema é usar setValor; o desconto deveria criar um novo objeto Pedido

D) O desconto deveria ficar no Repository, que grava o valor final do pedido

E) O código está correto, pois o Controller pode aplicar regras simples sem o Service

08 Em um ambiente IaaS, o provedor gerencia toda a infraestrutura física, incluindo hardware e rede física. **1,0 pt**

Quais elementos são de responsabilidade EXCLUSIVA do usuário nesse modelo?

A) Energia do datacenter, middleware e configuração de rede virtualizada

B) Sistema operacional, aplicações e rede virtual configurada

C) Infraestrutura física, banco de dados e atualizações do sistema operacional

D) Hipervisor, aplicações instaladas e segurança física do datacenter

E) Hardware físico, hipervisor e sistema operacional dos servidores

09 Considere o seguinte método em um Controller Spring: **1,0 pt**

```
@GetMapping("/produtos/{id}")
public ResponseEntity<Produto>
buscarProduto(@PathVariable Long id) {
    Produto produto =
    produtoService.buscarPorId(id);
    if (produto == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(produto);
}
```

Qual é o comportamento desse método?

A) Cria um novo produto com o id informado na URL

B) Remove do banco o produto com o id informado

C) Lista todos os produtos disponíveis no banco

D) Atualiza o produto e devolve o resultado com 200
E) Busca um produto pelo id e devolve 404 se não existir

10 Uma API precisa alterar só o e-mail de um usuário, sem enviar os demais campos. Qual método HTTP é o mais adequado? **1,0 pt**

A) PUT
B) GET
C) DELETE
D) POST
E) PATCH

11 O que diferencia microsserviços de SOA? **1,0 pt**

A) Microsserviços não usam contratos na comunicação entre serviços
B) A SOA é exclusiva para aplicações web, e microsserviços, para mobile
C) A SOA promove serviços menores que os dos microsserviços
D) Microsserviços favorecem deploy independente e serviços menores
E) A SOA é voltada apenas a grandes corporações com sistemas legados

12 Observe a estrutura de projeto abaixo: **1,0 pt**

```
com.exemplo.projeto
+-- controller
|   +-- UsuarioController.java
+-- model
|   +-- Usuario.java
+-- service
|   +-- UsuarioService.java
+-- repository
    +-- UsuarioRepository.java
```

Qual camada deve conter as regras como validação de CPF ou verificação de saldo?

A) service
B) model
C) controller
D) view
E) repository

13 Considere uma violação do padrão MVC onde a lógica de negócio está na camada Controller. Descreva os problemas que isso causa no longo prazo e como refatorar corretamente essa situação. **1,0 pt**

14 Associe cada anotação JPA da Coluna A à sua descrição na Coluna B. **1,0 pt**
Atenção: a Coluna B possui mais opções do que itens na Coluna A. Nem todas as opções serão utilizadas.

Coluna A:
1. @Entity
2. @Table
3. @Id
4. @GeneratedValue
5. @Column

Coluna B:
a) Especifica o nome e o schema da tabela no banco de dados
b) Marca a classe como entidade persistível gerenciada pelo JPA
c) Define a estratégia de geração automática do valor do campo
d) Indica que o campo é a chave primária da entidade
e) Permite configurar nome, tipo e restrições de um campo
f) Mapeia um relacionamento de coleção entre entidades
g) Indica que o campo não pode ser nulo na camada de negócio

15 Em qual destas situações o uso de um DTO é mais indicado? **1,0 pt**

A) Definir as rotas HTTP que o Controller vai expor para os clientes
B) Gravar no banco os dados de uma tabela, no lugar da classe anotada com @Entity
C) Devolver ao cliente só parte dos campos da entidade, sem expor dados internos
D) Concentrar as regras de negócio que hoje estão espalhadas pelo Service
E) Substituir o Repository nas consultas ao banco de dados

16 Uma empresa brasileira armazena dados pessoais de seus clientes em servidores de um provedor de nuvem localizado nos Estados Unidos, sem nenhuma cláusula contratual específica sobre localização e tratamento desses dados. **1,0 pt**

Qual desafio regulatório essa situação levanta diretamente?

A) Possível violação da LGPD, que exige garantias para transferir dados pessoais ao exterior

B) Risco de vendor lock-in pela ausência de cláusulas de portabilidade dos dados

C) Quebra contratual com o provedor por armazenar dados em solo estrangeiro

D) Possível violação da GDPR, que regula dados de cidadãos em servidores americanos

E) Violação da LGPD por ausência de criptografia nos dados durante a transferência

17 Considere a interface abaixo: **1,0 pt**

```
@Repository
public interface ClienteRepository extends
JpaRepository<Cliente, Long> {
}
```

O que os dois parâmetros genéricos <Cliente, Long> representam nessa declaração de JpaRepository<T, ID>?

A) Cliente é o Service que usa o repositório, e Long é o tipo do id

B) Cliente é a tabela do banco, e Long é o número máximo de registros

C) Cliente é a entidade gerenciada, e Long é o tipo da chave primária (@Id)

D) Cliente é a entidade gerenciada, e Long é o tipo de retorno do count()

E) Cliente é o DTO devolvido pelos métodos, e Long é o tipo do id desse DTO

18 Analise a classe: **1,0 pt**

```
@Service
public class PedidoService {

    @Autowired
    private PedidoRepository pedidoRepository;

    @Autowired
    private EmailService emailService;
}
```

a) Reescreva a classe usando injeção de dependência por construtor.
b) Cite duas vantagens dessa mudança, pensando nos testes e na manutenção.

19 Um banco decide operar sua própria infraestrutura de servidores internamente, mesmo assumindo um custo operacional significativamente mais alto do que contratar um provedor de nuvem pública. **1,0 pt**

Qual justificativa fundamenta essa escolha?

A) Maior controle técnico e redução dos custos operacionais a longo prazo

B) Melhor desempenho nas transações e escalabilidade automática do sistema

C) Controle sobre dados sensíveis e conformidade com regulações do setor

D) Menor latência nas transações e facilidade de expandir a infraestrutura

E) Eliminação de fornecedores externos e maior escalabilidade da infraestrutura

20 Em uma aplicação Spring Boot, qual é a diferença entre as anotações @Controller e @RestController? Em que situações cada uma deve ser usada? Ilustre com um exemplo de código. **1,0 pt**

Cartão-Resposta

Arquitetura Orientada a Servicos · Prova · 2026

ALUNO(A)	TURMA	Nº

01	—					02	A	B	C	D	E
03	A	B	C	D	E	04	A	B	C	D	E
05	A	B	C	D	E	06	A	B	C	D	E
07	A	B	C	D	E	08	A	B	C	D	E
09	A	B	C	D	E	10	A	B	C	D	E
11	A	B	C	D	E	12	A	B	C	D	E
13	—					14	—				
15	A	B	C	D	E	16	A	B	C	D	E
17	A	B	C	D	E	18	—				
19	A	B	C	D	E	20	—				

Preencha completamente a bolha correspondente à alternativa escolhida, usando caneta azul ou preta. Questões dissertativas (—) são corrigidas diretamente na folha da prova. Não rasure: uma única marcação por questão.

Gabarito: Arquitetura Orientada a Servicos

ARQUITETURA ORIENTADA A SERVICOS · PROVA · 2026

CONFIDENCIAL · PROFESSOR

A ordem das questões muda em cada versão. Cada tabela corresponde ao código impresso no canto inferior do cartão-resposta. As dissertativas (Dissert.) são detalhadas em “Respostas esperadas”, identificadas pelo resumo do enunciado.

TM-QW4HE

QUESTÃO	RESPOSTA
01	Dissert.
02	D
03	E
04	B
05	A
06	D
07	B
08	B
09	E
10	E
11	D
12	A
13	Dissert.
14	Dissert.
15	C
16	A
17	C
18	Dissert.
19	C
20	Dissert.

RESPOSTAS ESPERADAS: DISSERTATIVAS

Escreva o Javadoc do método abaixo, documentando o que ele…

```
/**
 * Busca um produto pelo id.
 *
 * @param id identificador do produto
 * @return o produto encontrado, convertido em DTO
 * @throws ProdutoNaoEncontradoException se não existir produto com esse id
 */
```

O Javadoc documenta o código para quem mantém o sistema. O contrato da API, para quem a consome, fica no OpenAPI/Swagger.

Considere uma violação do padrão MVC onde a lógica de…

Problemas: dificuldade de testes unitários (Controller mistura HTTP e regras de negócio), violação do princípio de responsabilidade única, reuso impossibilitado (regra não pode ser chamada por outro contexto sem HTTP), manutenção difícil e código duplicado. Refatoração: extrair a lógica de negócio para uma classe Service anotada com @Service, deixando o Controller responsável apenas por receber a requisição, delegar ao Service e retornar a resposta. Isso permite testar o Service independentemente.

Associe cada anotação JPA da Coluna A à sua descrição na…

@Entity → b | @Table → a | @Id → d | @GeneratedValue → c | @Column → e | As opções f e g são distratores e não possuem correspondência.

Analise a classe: @Service public class PedidoService {…

a)

```
@Service
public class PedidoService {

    private final PedidoRepository pedidoRepository;
    private final EmailService emailService;

    public PedidoService(PedidoRepository pedidoRepository, EmailService emailService) {
        this.pedidoRepository = pedidoRepository;
        this.emailService = emailService;
    }
}
```

Com um único construtor, o Spring injeta as dependências sem precisar de @Autowired.
b) Nos testes, basta passar mocks no construtor, sem reflection nem contexto do Spring. As dependências ficam explícitas na assinatura, podem ser final, e a classe nunca existe sem elas.

Em uma aplicação Spring Boot, qual é a diferença entre as…

@Controller é usada em aplicações MVC tradicionais onde os métodos retornam nomes de views (templates Thymeleaf, JSP). @RestController combina @Controller e @ResponseBody, serializando automaticamente o retorno dos métodos para JSON/XML, sendo usada em APIs REST. Exemplo: @RestController public class ProdutoController { @GetMapping("/produtos") public List<Produto> listar() { return service.listar(); } }

Criado com **TestMaker**